Loop Unrolling

Performance Optimization & Parallelism Guide

Loop unrolling is a programming optimization technique that increases a program's execution speed by reducing the overhead associated with loops. Instead of executing a loop 100 times, the code is rewritten (either manually by the programmer or automatically by the compiler) to execute more operations per iteration, reducing the total number of iterations.

1. CODE COMPARISON

Standard Loop (Before)

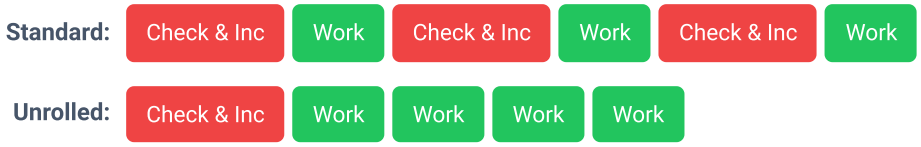
```
        for (int i = 0; i < 100; i++)
{ sum += array[i]; }
```

Unrolled Loop (After, Factor of 4)

```
        for (int i = 0; i < 100; i +=
4) { sum += array[i]; sum +=
array[i+1]; sum += array[i+2]; sum +=
array[i+3]; }
```

2. HOW IT IMPROVES PERFORMANCE

In a standard loop, the CPU spends a significant amount of time just managing the loop itself (checking the condition, incrementing the counter, and jumping back to the start). By unrolling the loop, we minimize this "wasted" time.



3. ENHANCING PARALLELISM

> **Primary Enhancement: Instruction-Level Parallelism (ILP)**
> Modern processors can execute multiple instructions at the exact same time if those instructions do not depend on each other.

When a loop is unrolled, a large block of independent instructions is exposed to the CPU. Because there are no immediate branch instructions (jumps) interrupting the flow, the processor's pipeline can easily grab multiple `sum += array[...]` instructions and process them simultaneously. It turns sequential, interrupted steps into a smooth, parallel pipeline of operations.

4. POTENTIAL DRAWBACKS (TRADE-OFFS)